

Vistas (Views): consultas que cualquiera puede usar

Hasta ahora todo lo que hemos construido en el curso — los JOINS, las subqueries, las CTEs — requiere que la persona que quiera ver los datos sepa escribir SQL. Y eso está bien cuando eres tú el que consulta la base de datos. Pero en la realidad de cualquier empresa, hay gente que necesita ver información todos los días y no tiene ni idea de cómo escribir una consulta. El equipo comercial, el área de marketing, la gerencia.

Pensemos en TiendaLatam. El equipo comercial necesita ver las ventas por país todos los días. Podrías pasarles la query con cuatro JOINS y un GROUP BY, pero no van a saber qué hacer con eso. O podrías crear algo llamado una **vista**, ponerle el nombre `VW_VentasPorPais`, y que ellos simplemente escriban `SELECT * FROM VW_VentasPorPais` como si fuera una tabla más. Eso es exactamente lo que vamos a ver en esta clase.

¿Qué es una vista y qué no es?

Una vista es una consulta guardada con un nombre. Eso es todo. No es una tabla nueva, no almacena datos, no ocupa espacio con registros. Lo que guarda es **la definición de la consulta** — el SELECT con sus JOINS, sus filtros, sus agrupaciones. Cada vez que alguien consulta la vista, SQL Server ejecuta esa query en ese instante y le devuelve el resultado actualizado.

Esto es importante entenderlo bien porque es la confusión más común. La vista no es una foto estática de los datos. Es una ventana en tiempo real. Si hoy se registran nuevas ventas en la base de datos, mañana cuando alguien consulte la vista esas ventas ya van a estar ahí, sin que nadie tenga que hacer nada.

¿Y por qué esto es útil? Por tres razones que se complementan entre sí.

La primera es **abstracción**. El usuario ve columnas con nombres claros — NombrePais, VentasTotales, TicketPromedio — sin tener que saber que detrás hay cuatro tablas cruzadas con JOINS. Para él, es como si fuera una tabla simple.

La segunda es **seguridad**. Puedes darle a alguien acceso solo a la vista, sin darle acceso a las tablas originales. El analista de ventas puede consultar `VW_VentasPorPais` todo lo que quiera, pero no puede meterse a la tabla `Pedidos` ni a la tabla `Clientes` directamente. Este patrón — exponer vistas, ocultar tablas —

es la forma estándar de implementar lo que se llama el principio de mínimo privilegio en bases de datos.

Y la tercera es **mantenimiento**. Si mañana cambia la lógica de negocio — por ejemplo, ahora quieren que el reporte incluya la venta mínima y máxima además del promedio — actualizas la vista en un solo lugar y todos los que la usan empiezan a ver la nueva definición automáticamente. No tienes que ir usuario por usuario avisándoles que cambió la query.

Crear, usar y modificar vistas

Crear una vista es sorprendentemente sencillo. La sintaxis es `CREATE VIEW`, el nombre que le quieras dar, la palabra `AS`, y después la consulta completa. Cierras con `GO` y listo. La vista queda registrada en la base de datos y aparece en el explorador de objetos de SQL Server, dentro de la carpeta "Vistas".

Una buena práctica que vale la pena adoptar desde ahora es ponerle el prefijo `VW_` a todas tus vistas. Cuando la base de datos crece y tienes decenas de objetos — tablas, vistas, procedimientos almacenados — ese prefijo te permite identificar rápidamente qué es una vista y qué no. Es un estándar de la industria que te va a ahorrar tiempo.

Una vez creada, usar la vista es exactamente igual a usar una tabla. Puedes hacer `SELECT * FROM VW_VentasPorPais`, puedes ponerle filtros con `WHERE`, puedes agruparla, ordenarla, cruzarla con otras tablas o con otras vistas. Todo lo que sabes hacer con tablas aplica acá.

¿Y si necesitas cambiar la definición? Para eso existe `ALTER VIEW`. La lógica es la misma que con `CREATE`, pero en vez de crear una vista nueva, reemplazas la definición de una que ya existe. En la clase vimos cómo la vista original tenía ventas totales y ticket promedio, y con un `ALTER` le agregamos venta mínima y venta máxima. Después de ejecutar el `ALTER`, cualquier persona que consulte la vista inmediatamente ve las nuevas columnas.

Si en algún momento la vista ya no es necesaria, la eliminas con `DROP VIEW IF EXISTS` y el nombre. Desaparece de la base de datos y del explorador de objetos.

Y un detalle útil: si quieres ver cuál es la consulta que está detrás de una vista — porque la creó otra persona o porque ya no te acuerdas — tienes dos caminos. Uno es ir al explorador de objetos, hacer clic derecho sobre la vista y buscar la opción de

"Script View as". El otro es ejecutar `EXEC sp_helptext 'NombreDeLaVista'`, que te devuelve la definición completa como resultado de una consulta. Ambos te muestran lo mismo, es cuestión de preferencia.

Lo que las vistas no pueden hacer

Las vistas son muy útiles, pero no son magia. Tienen limitaciones que conviene conocer de entrada para no encontrarse con sorpresas.

No pueden tener `ORDER BY` por sí solas — a menos que uses `TOP` o `OFFSET/FETCH`. Esto es una restricción de SQL Server: la vista define qué datos se ven, pero el orden lo decide la query que la consulta.

No pueden recibir parámetros. Si necesitas algo como "muéstrame las ventas de este país específico" donde el país cambia según quién consulta, la vista no puede hacerlo por sí sola — tienes que pasarle el filtro desde afuera con un `WHERE`, o usar funciones de tabla, que las vamos a ver más adelante en el curso.

No puedes hacer `INSERT`, `UPDATE` o `DELETE` directamente sobre vistas que tienen `GROUP BY`, `JOINS` complejos o columnas calculadas. Las vistas que vimos acá son de solo lectura, y en la práctica la mayoría lo son.

Y quizás lo más importante: como la vista no almacena datos sino que ejecuta la consulta cada vez, **si la query subyacente es lenta, la vista también va a ser lenta**. No hay magia de rendimiento acá. La vista es exactamente tan rápida como la consulta que tiene adentro.

Sobre ese último punto hay una excepción interesante: las vistas indexadas, que sí pueden almacenar su resultado. Pero eso es tema de la próxima clase.

Desafío de cierre

Crea tres vistas para TiendaLatam. La primera, `VW_VentasPorPais`, con las ventas agrupadas por país, año y mes. La segunda, `VW_ProductosTopVentas`, con los veinte productos que más unidades vendieron. Y la tercera, `VW_ClientesSinPedido`, con los clientes que nunca realizaron un pedido (acá te va a servir el patrón de `LEFT JOIN + WHERE IS NULL` que aprendimos en la Clase 1). Una vez creadas, verifica que puedes consultarlas como si fueran tablas normales. Comparte tus resultados en los comentarios de la clase.